

Mini Lecture:

Garbage Collection

CIS531 — Fall 2025, Syracuse

Kristopher Micinski



Memory Management: the Runtime Perspective

- * Every language needs *some* story for memory management
 - * Stack-allocate everything? Everything except globals?
 - * Stack allocation is simple, as every object's lifetime is dictated by its corresponding function's dynamic extent (freed upon return)
- * In general, we want heap-allocated data
 - * The moment we have heap-allocated data, we need some story for how we manage memory
- * In C, we have **manual** memory management via malloc / free
 - * You're in charge of ensuring you free memory, never double free it, etc.
 - * Very cumbersome and tricky to get right
 - * So often, this is **The** source of security vulnerabilities!

Automatic Memory Management

- * Aside from manual memory management, we have **automatic** memory management: no need to manually free objects
- * This is so much easier, because now we don't have to think about the dynamics of allocating / freeing memory
- * Memory management is not really relevant to the functional correctness of the code, so it's **especially frustrating** to think about
- * We'll talk about **Garbage Collection**, the most common form of automatic MM
 - * GC has some upsides, but it is generally a *heavy* imposition on the runtime
- * Important caveat: Rust does **not** have a garbage collector, instead it has a significantly more complicated system for reasoning about lifetimes

Reference Counting

- * We'll start by talking about something simple that you can implement in any language: reference counting
- * Basic idea: for every heap-allocated object, we attach a reference count, the number of distinct, currently-life pointers that exist to that object
- * When object is malloc'd, set its reference count to 1
- * When a pointer is copied, bump the reference count by 1
 - * Can use *copy constructors* in C++ to achieve this seamlessly
- * When pointer goes out of scope (e.g., function returns, stack deallocated), decrement reference count by 1
- * Each time you decrease the reference count, check if it hits 0
 - * When it hits 0, you free it
- * **Must** ensure we instrument **every** pointer copy, creation, etc.

Mark and Sweep GC

- * First concession: at various points, we will interrupt the program to **stop the world**, to run the garbage collector, which systematically scans the heap for obviously-dead stuff that can be freed
- * Mark/Sweep is the easiest one to understand:
 - * Start with a **root set** of pointers to *potentially alive* (i.e., not obviously-dead) objects. This is generally the stack and all of the globals, since those things can potentially be accessed at some later point in the program
 - * At the beginning of a GC phase, mark all objects as dead. Heap-allocated objects need to have some bits for the GC to use, typically included in the object header (you, the designer, controls object layout)
 - * Next, take everything in the root set and mark it as live, then consider everything pointed to by those objects: follow the pointers and mark **those** objects as live, too. Do this to a fixed-point (i.e., until you can't anymore)
 - * Last, sweep through the heap, free everything not marked as live

Downsides to Mark / Sweep

- * **Full** heap traversal, very heavyweight, long stop-the-world pauses
 - * If done without subsequent heap compaction, we generally end up with heap fragmentation
 - * Poor locality for surviving objects: we'll see some alternatives later
 - * Difficult to parallelize sweeping in an efficient manner
 - * Work proportional to *total heap size* **not** live data
-
- * Mark/sweep is a very popular concept that is spiritually the core of a lot of GC algorithms, but these downsides mean that in practice we often do something a bit more complicated / different

Copying Collectors: Cheney's Semispace Algorithm

- * Basic idea: use two heaps, one heap is always free (uses 2x the space!)
 - * Split into from-space (copying from) and to-space (copying to)
- * Stop the world and start at roots: traverse reachable data in a breadth-first manner via two pointers, the *scan pointer* and *free pointer*
 - * Scan each live object into to-space, then scan the copy
- * Perform breadth-first traversal, walking through objects in to-space in the order in which they were copied
- * Use two pointers: scan pointer (walks through to-space) and free pointer (allocates new copied objects into the to-space)
- * On GC: (1) set scan = free = start of to-space, (2) copy root objects into to-space, assigning forward pointers, (3) while scan < free: for each pointer in the object at scan, if not yet copied (look at high bits), copy and update ref, (4) advance scan pointer
- * Finally, swap spaces: to-space becomes active heap, from-space ready for next use
- * Great idea! Simple, fast, naturally compacts! But requires 2x the space :(

Mark/Compact GC

- * GC that moves live objects, in place to eliminate fragmentation
 - * Produces dense, contiguous live data (good locality) without overhead of Cheney's
- * Super fast (bump pointer) allocation after compaction
- * Idea: (1) mark live objects, (2) compute new locations based on marking, (3) relocate objects to new addresses, (4) update pointers to relocated objects
- * Pros: produces compact heaps, good for locality
- * Cons: requires fixing up pointers, generally slower than copying-based collectors
- * Must handle pinned objects (native code holds raw pointers), handling large objects is hard for various reasons (expensive to copy, hard to fit back into compacted areas, etc.)

Generational GC

- * Key idea: divide heap into **generations**
 - * Long-lived objects, etc..., and finally the “nursery” (just allocated) generation
 - * When objects first allocated, they get put in the nursery generation
- * Scan the nursery frequently; evacuate survivors of nursery by copying into an older generation. Most objects die young: copying is cheap/fast
 - * Less frequently, do full mark/sweep or mark/compact on the major (old) heap
- * Works because lifecycles generally have a long-tail distribution, with many short-lived objects and fewer, longer-living objects
- * V8 (Chrome): copying for minor heap, mark/sweep or mark/compact based on memory pressure and per-page fragmentation

Intergenerational References

- * Pointers from old to young, or young to old object, crossing between generations at any depth
- * Mutator thread (GC collector/worker thread) might create these at any time
- * If an old object points to a young object, GC must treat it as a root: otherwise, young objects may be incorrectly reclaimed
- * GC only visits minor heap, need mechanism to record cross-gen refs
- * Use **write barrier** to detect stores into old objects
 - * On every pointer store, check `is_old(obj_being_written)` AND `is_young(value_being_stored)` — if so, record old -> young edge in card table
- * Card table is a bitmap-like array used by generational GC: heap divided into fixed-sized “cards” (e.g., 256-512 bytes), card table stores one bit per card
- * “Anything in this card might contain a pointer to the nursery”
- * Upon minor GC, dirty cards are scanned, treated as root set

Incremental GC

- * Stopping the world is frustrating and we would prefer to avoid it if possible
- * But if the program keeps running, concurrent with the collector, it can modify arbitrary pointers and ***easily break necessary GC invariants***
- * If we want incremental GC, we can use **tri-color** GC; we maintain three sets
 - * White set (condemned set): set of objects unvisited, up for grabs to be reclaimed at the end of this cycle
 - * Grey set: all objects reachable from roots, but not yet scanned for references to white objects. Can't be GC'd, will be moved to black after full scan
 - * Black set: definitely no outgoing references to objects in the white set, fully done processing
- * Initially, everything reachable from the root set starts grey, everything else starts white. Then, to a fixed point:
 - * Pick an grey object, move each white object that o references to the grey set, finally move o to the black set; repeat this process until grey set empty
 - * Tri-color invariant: no black object references any white object (whites collected)

See a demo: https://en.wikipedia.org/wiki/Tracing_garbage_collection#/media/File:Animation_of_tri-color_garbage_collection.gif

Modern Multithreaded GC: Key Themes

- * Parallel mark + sweep/compact phases
- * Concurrent phases to minimize STW time
- * Region-based heaps for scalable parallelism
- * Writer barriers (always) + optional read barriers
- * NUMA-aware allocation / evacuation
- * Incremental work scheduling across threads

Main thread total young generation garbage collection time [ms]

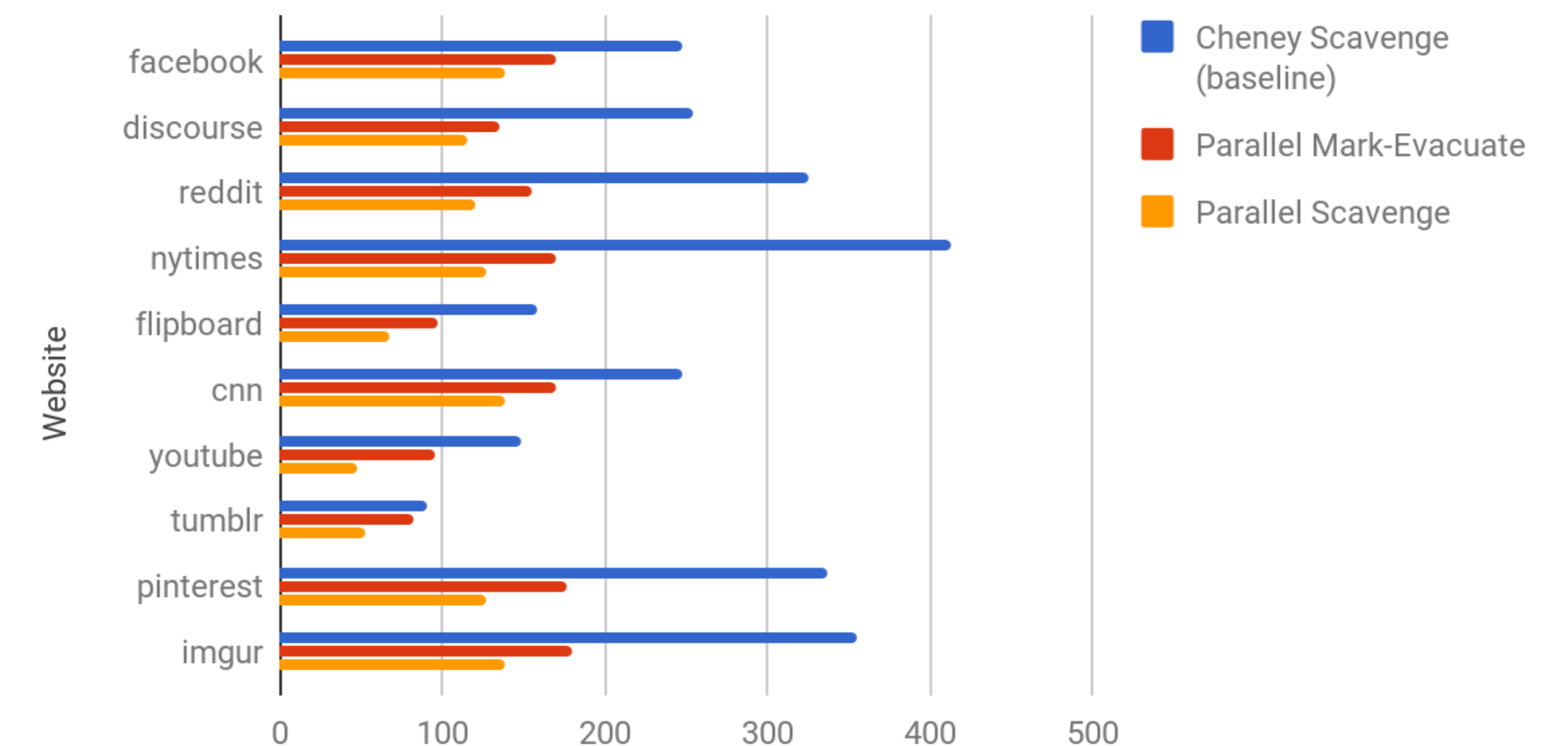


Photo from the v8 blog

<https://v8.dev/blog/orinoco-parallel-scavenger>